

Reconstrucción de un Proceso Scrum y DevOps mediante Arqueología de Repositorios: Análisis del Caso Mindustry

Sandra Gabriela Aliaga Chaiña

Escuela Profesional de Ingeniería de
Sistemas
Universidad Nacional de San Agustín
de Arequipa
Arequipa, Perú
saliagac@unsa.edu.pe

Tania Luz Ayque Puraca

Escuela Profesional de Ingeniería de
Sistemas
Universidad Nacional de San Agustín
de Arequipa
Arequipa, Perú
tayque@unsa.edu.pe

Karla Miluska Bedregal Coaguila

Escuela Profesional de Ingeniería de
Sistemas
Universidad Nacional de San Agustín
de Arequipa
Arequipa, Perú
kbedregalc@unsa.edu.pe

Rodrigo Fabricio Layme Salas

Escuela Profesional de Ingeniería de
Sistemas
Universidad Nacional de San Agustín
de Arequipa
Arequipa, Perú
rlaymes@unsa.edu.pe

Jack Franco Llaique Chullunquia

Escuela Profesional de Ingeniería de
Sistemas
Universidad Nacional de San Agustín
de Arequipa
Arequipa, Perú
jllaique@unsa.edu.pe

**Joaquin Alejandro Quispe
Bedregal**

Escuela Profesional de Ingeniería de
Sistemas
Universidad Nacional de San Agustín
de Arequipa
Arequipa, Perú
jquispebed@unsa.edu.pe

Resumen—La enseñanza de metodologías ágiles y prácticas DevOps enfrenta una dificultad recurrente: la mayoría de los proyectos académicos parten de cero y no permiten observar cómo evoluciona un sistema real a lo largo de años de desarrollo. Este trabajo propone y aplica una estrategia de ingeniería inversa, denominada método del cangrejo, sobre Mindustry, un videojuego de estrategia de código abierto con más de nueve años de historia. Partiendo del estado actual del repositorio (versión V8) se reconstruye retrospectivamente un backlog Scrum de cuatro sprints y se contrasta, commit a commit, con las decisiones técnicas realmente adoptadas por el proyecto. En paralelo se implementa una infraestructura de Integración y Entrega Continuas funcional sobre un fork del equipo, que evoluciona en tres niveles de madurez: verificación automatizada, entrega automatizada y aseguramiento de calidad con cobertura de pruebas. Los resultados muestran que las decisiones técnicas reales del proyecto (separación temprana de un módulo core agnóstico de plataforma, inversión de dependencias en la capa de red, postergación de plataformas y herramientas no esenciales) coinciden con las que la metodología ágil habría prescrito, y que la abstracción temprana de interfaces fue el factor que hizo posibles migraciones arquitectónicas posteriores sin reescritura del núcleo. Se concluye que la arqueología de repositorios es un recurso didáctico válido para evidenciar principios de diseño evolutivo y para justificar prácticas DevOps sobre una base funcional preexistente.

Palabras clave—Scrum, DevOps, integración continua, entrega continua, minería de repositorios de software, evolución del software, deuda técnica, videojuegos de código abierto.

I. INTRODUCCIÓN

El presente artículo sintetiza el proyecto final del curso Ingeniería y Procesos de Software (2026-A), cuyo objetivo fue aplicar la metodología Scrum integrada con prácticas

DevOps sobre un sistema de software real y de gran madurez. El caso de estudio es Mindustry, un videojuego de estrategia y gestión de recursos de código abierto desarrollado por Anuken bajo licencia GPL, con más de nueve años de desarrollo continuo y una versión actual identificada como V8.

La dificultad central que motiva este trabajo es de índole didáctica. Los estudios empíricos sobre metodologías ágiles coinciden en que sus beneficios se manifiestan a lo largo de múltiples iteraciones y ante requisitos cambiantes [1], y en que DevOps solo cobra sentido cuando existe una base de software sobre la cual automatizar la integración y la entrega [2]. Sin embargo, un proyecto académico que parte de cero rara vez alcanza la escala temporal necesaria para observar estos fenómenos. Para superar esta limitación se adopta una estrategia de ingeniería inversa sobre el historial de control de versiones, en línea con la tradición de la minería de repositorios de software, que trata la historia de commits como una fuente empírica de evidencia sobre las decisiones de ingeniería [3].

La estrategia, denominada por el equipo método del cangrejo, consiste en partir del estado actual del repositorio y retroceder en el tiempo para reconstruir cómo debió haberse planificado y ejecutado el desarrollo con Scrum y DevOps desde el primer commit. Esta perspectiva se apoya en el hecho, establecido desde los trabajos clásicos sobre evolución del software, de que todo sistema en uso real está sometido a un cambio continuo y a una complejidad creciente que dejan huellas observables en su historia [4]. El propósito no es reescribir el software, sino leer su historia como evidencia y construir sobre un fork del equipo una infraestructura de Integración y Entrega Continuas (CI/CD) genuinamente funcional.

Este trabajo persigue tres objetivos. Primero, reconstruir un backlog Scrum coherente distribuido en cuatro sprints y contrastarlo, con evidencia de commits, contra las decisiones técnicas realmente adoptadas por Mindustry. Segundo, implementar y documentar una infraestructura CI/CD sobre el fork del equipo, incluyendo análisis de calidad y un release final automatizado. Tercero, extraer lecciones metodológicas sobre diseño evolutivo, inversión de dependencias y gestión de deuda técnica transferibles al trabajo profesional con código heredado. El resto del artículo se organiza en trabajos relacionados, marco teórico, enfoque propuesto, metodología, resultados, discusión y conclusiones.

II. TRABAJOS RELACIONADOS

A. Estudios empíricos sobre Scrum y metodologías ágiles

La revisión sistemática de Dyba y Dingsoyr consolidó la evidencia empírica temprana sobre desarrollo ágil y estableció que sus beneficios reportados dependen fuertemente del contexto y de la disciplina del equipo [1]. Más recientemente, Verwijs y Russo propusieron y validaron, a partir de datos de cerca de 2.000 equipos Scrum, una teoría de efectividad que identifica la capacidad de respuesta, la mejora continua, la autonomía del equipo y el apoyo de la gestión como factores determinantes [5]. Ambos trabajos justifican tratar la retrospectiva y el refinamiento del backlog como mecanismos centrales, y no accesorios, del proceso ágil.

B. DevOps: definiciones y prácticas

El estudio de mapeo sistemático de Jabbari et al. es una referencia canónica para caracterizar DevOps, pues identificó sus componentes recurrentes en la literatura y su relación con el desarrollo ágil, del que constituye una extensión orientada a la operación [2]. Mishra y Otaíwi, por su parte, analizaron la relación entre DevOps y la calidad del software y encontraron que sus prácticas de automatización tienden a incrementar la frecuencia y la fiabilidad de las entregas [6]. Estos trabajos enmarcan la CI/CD como el núcleo operativo de DevOps.

C. Integración y entrega continuas

La evidencia empírica sobre CI proviene de estudios a gran escala en proyectos de código abierto. Hilton et al. caracterizaron el uso, los costos y los beneficios de la integración continua sobre decenas de miles de proyectos de GitHub, mostrando que su adopción está asociada a lanzamientos más frecuentes [7]. Vasilescu et al. hallaron que los equipos que adoptan CI integran más contribuciones externas y detectan más defectos sin una pérdida observable de calidad [8]. Estos resultados fundamentan la decisión de construir el pipeline CI/CD como columna vertebral del proyecto.

D. Minería de repositorios y evolución del software

La minería de repositorios de software ofrece el sustento metodológico del método del cangrejo. Kagdi et al. presentaron una taxonomía de enfoques para analizar repositorios en el contexto de la evolución del software [9], mientras que Kalliamvakou et al. advirtieron sobre las promesas y los riesgos de extraer conclusiones a partir de

datos de GitHub, un conjunto de cautelas metodológicas que este trabajo tiene en cuenta [3]. En el plano teórico, las leyes de evolución del software de Lehman describen el cambio continuo y la complejidad creciente de los sistemas en uso [4], cuya vigencia ha sido revisada sistemáticamente por Herraiz et al. [10].

E. Ingeniería de software en videojuegos

El desarrollo de videojuegos presenta particularidades frente al desarrollo de software convencional. Murphy-Hill et al. documentaron, mediante entrevistas y encuestas, diferencias sustanciales en la gestión de calidad, la evolución de artefactos y las habilidades requeridas [11]. Pascarella et al. confirmaron estas diferencias en el contexto de código abierto y observaron que los equipos de juegos organizan sus recursos y perciben la reutilización y las pruebas automatizadas de manera distinta [12]. Estos hallazgos son pertinentes porque el objeto de estudio es precisamente un videojuego de código abierto.

F. Deuda técnica y refactorización

La deuda técnica proporciona el vocabulario para interpretar las decisiones de compromiso observadas en el historial. El mapeo sistemático de Li, Avgeriou y Liang consolidó su definición y gestión [13], y Kruchten et al. la elevaron de metáfora a marco de análisis práctico [14]. Behutiye et al. estudiaron específicamente la deuda técnica en el contexto ágil [15], y Rios et al. ofrecieron un estudio terciario sobre tipos y estrategias de gestión [21]. Respecto de la refactorización, el estudio de campo de Kim, Zimmermann y Nagappan en Microsoft mostró que la refactorización reduce dependencias entre módulos pero conlleva costos y riesgos reales [16], mientras que Murphy-Hill, Parnin y Black caracterizaron como los desarrolladores refactorizan en la práctica [17].

III. MARCO TEÓRICO

A. Scrum y desarrollo ágil

Scrum es un marco iterativo e incremental que organiza el trabajo en sprints de duración fija y en torno a un backlog priorizado. En este trabajo se emplean sus artefactos (backlog de producto, backlog de sprint y tablero Kanban), sus eventos (planificación, revisión y retrospectiva) y sus roles, entendidos como instrumentos para gestionar la incertidumbre y priorizar la entrega de valor incremental [1], [5].

B. DevOps y el ciclo CI/CD

DevOps se entiende como la extensión de los principios ágiles hacia la operación, con la automatización como práctica central [2]. La Integración Continua compila y prueba automáticamente cada cambio integrado, reduciendo el riesgo de regresiones [7], [8]; la Entrega Continua extiende ese flujo hasta la generación automática de artefactos desplegables. La evidencia empírica asocia estas prácticas con mayor frecuencia de entrega y mejor detección de defectos [6], [7].

C. Minería de repositorios y arqueología de software

La minería de repositorios trata la historia de control de versiones como fuente de datos para comprender procesos

de ingeniería [3], [9]. El método del cangrejo aplica esta idea de forma dirigida: en lugar de agregar estadísticas sobre muchos proyectos, realiza una lectura interpretativa de un único repositorio para reconstruir su trayectoria de decisiones. Esta lectura se apoya en la noción de que los cambios de mantenimiento suelen agruparse y dejar patrones identificables en el historial [19].

D. Modularidad e inversión de dependencias

El principio de modularidad y ocultamiento de información, formulado por Parnas, sostiene que un sistema debe descomponerse de modo que cada módulo oculte una decisión de diseño susceptible de cambiar, minimizando el acoplamiento [18]. La inversión de dependencias es la aplicación de este principio a las fronteras del sistema: definir interfaces estables de las que dependa la lógica de negocio, en lugar de depender de implementaciones concretas. Este principio es clave para interpretar la arquitectura de Mindustry.

E. Deuda técnica y refactorización

La deuda técnica designa los compromisos de diseño que aportan un beneficio de corto plazo a costa de un costo futuro [13], [14]. Su gestión deliberada, mediante refactorizaciones planificadas, es un signo de madurez de proceso [16], [17]. En el análisis arqueológico, distinguir una refactorización que salda deuda técnica de una regresión es una competencia central que este trabajo busca desarrollar [20].

IV. ENFOQUE PROPUESTO

El enfoque propuesto sostiene que un proyecto de software maduro puede utilizarse como laboratorio de ingeniería inversa para enseñar Scrum y DevOps sin renunciar al rigor. La tesis es que las decisiones de un proyecto exitoso, aunque no haya empleado una metodología formal, tienden a coincidir con las que un proceso ágil habría prescrito, porque responden a las mismas presiones de evolución y complejidad crecientes descritas por Lehman [4], [10].

Operativamente, el enfoque parte del estado V8 del repositorio y reconstruye un backlog de cuatro etapas ordenadas por dependencia técnica: (1) núcleo, renderizado y bucle de juego; (2) portabilidad móvil, persistencia y edición de contenido; (3) red, mods y ecosistema; y (4) calidad, release final y cierre. Cada historia de usuario del backlog se valida contra los commits que la materializan, de modo que la secuencia hipotética prescrita por Scrum se contrasta con la secuencia real observada. Sobre el fork del equipo se construye además una infraestructura CI/CD que no es simulada, sino ejecutable, y que evoluciona en paralelo a la investigación.

V. METODOLOGÍA

A. Objeto de estudio y fork del equipo

El objeto de estudio es el repositorio de Mindustry en su versión V8 (abril de 2026). Todo el trabajo de infraestructura se realiza sobre un fork del equipo, con el fin de no interferir con el repositorio original y de disponer de permisos para configurar flujos de automatización. La

ejecución de la versión actual desde el fork proporciona evidencia visual del producto final.

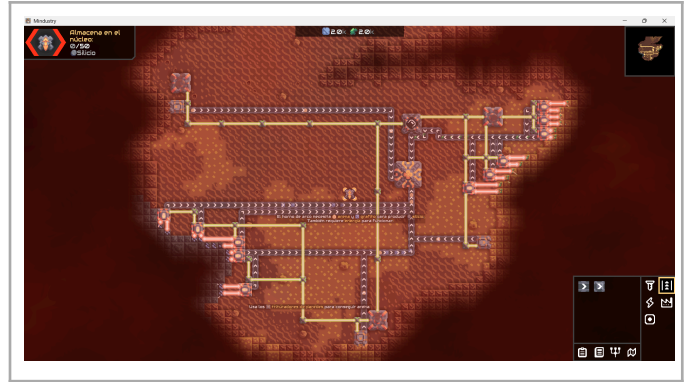


Fig 1. Mindustry V8 en ejecución desde el fork del equipo

B. Procedimiento de investigación arqueológica

La investigación arqueológica sigue un procedimiento reproducible por módulo. Primero se identifican los commits relevantes con consultas al historial (git log con filtros por ruta, fecha y palabra clave). Luego se reconstruye el estado del repositorio en un commit determinado mediante checkout o worktree, capturando la estructura de directorios y los archivos clave. Finalmente se analizan las decisiones técnicas que los commits revelan, interpretandolas a la luz de los principios de modularidad, inversión de dependencias y gestión de deuda técnica [18], [13]. Este procedimiento adopta las cautelas metodológicas propias de la minería de repositorios [3].

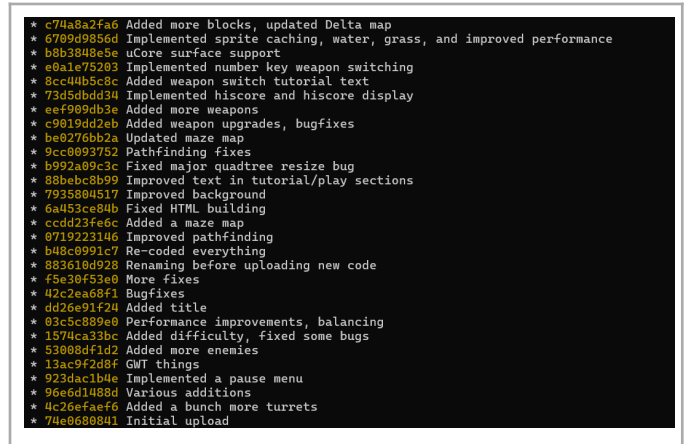
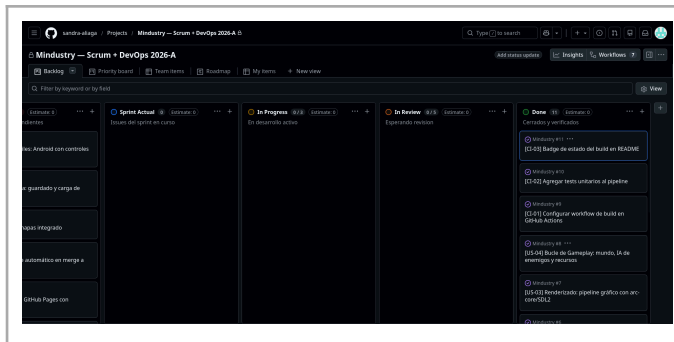


Fig 2. Salida de git log --online --graph del periodo fundacional.

C. Organización en sprints y backlog reconstruido

El backlog reconstruido se distribuye en cuatro sprints correspondientes a las cuatro etapas. Cada sprint define historias de usuario con puntos de historia estimados, roles Scrum rotativos (jefe de sprint, investigadores por módulo y responsables de CI/CD) y una retrospectiva. El tablero Kanban del proyecto registra el avance de cada historia hasta la columna Done, y los issues se cierran con trazabilidad hacia los commits de evidencia.



D. Implementación de la infraestructura CI/CD

La Integración Continua se implementa como un flujo de GitHub Actions que, ante cada push a la rama principal, configura el entorno con JDK 17, obtiene la dependencia Arc, habilita cache de Gradle, compila el proyecto y ejecuta la suite de pruebas. La Entrega Continua extiende ese flujo con la generación automática de artefactos y la creación de releases, y con el despliegue del sitio de documentación en GitHub Pages. En el sprint de cierre se incorpora un paso de análisis de calidad con cobertura de pruebas (JaCoCo) que interrumpe el pipeline si la cobertura cae por debajo de un umbral acordado, en línea con la evidencia sobre los beneficios de la CI [7], [8].

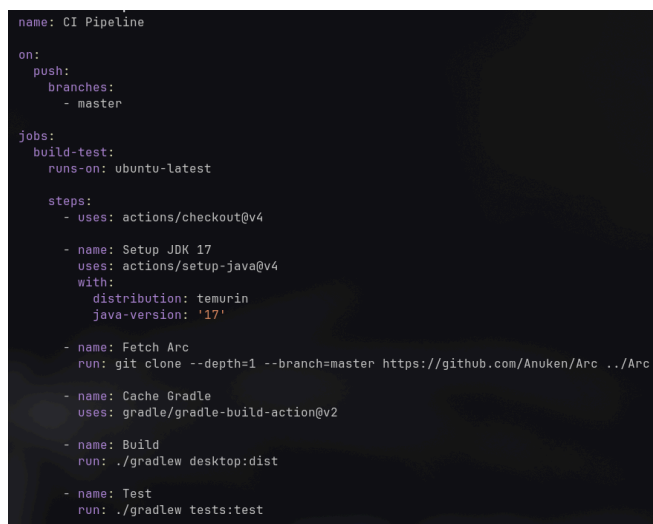


Fig. 4. Configuración del pipeline de integración continua en el archivo ci.yml.

VI. RESULTADOS

A. Sprint 1: núcleo, renderizado y bucle de juego

El análisis del periodo fundacional (abril a junio de 2017) confirma que la primera decisión arquitectónica fue aislar la lógica del juego en un módulo core agnóstico de plataforma, con los lanzadores desktop y html como únicos puntos de entrada iniciales. Esta separación, coherente con el principio de ocultamiento de información [18], permitió que el motor de reglas evolucionara sin depender de librerías nativas. Se identifican además optimizaciones tempranas de rendimiento (estructura espacial Quadtree para colisiones y caché de sprites estáticos) que evidencian una preocupación por la escalabilidad desde el MVP. La lógica de IA

(pathfinding A) ya existía en el primer commit, lo que confirma que el bucle jugable fue prioridad inicial. Sobre el fork se configuró la Integración Continua y se verificó su ejecución.

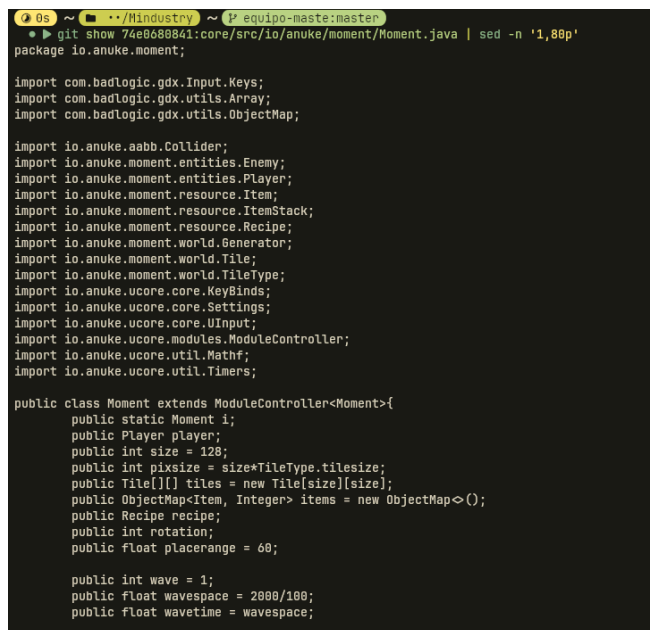


Fig. 5. Archivo principal *Moment.java* del módulo *core* en el commit fundacional 74e0680841.

B. Sprint 2: portabilidad móvil, persistencia y edición

El soporte Android aparece el 29 de junio de 2017, dos meses después del primer commit, lo que evidencia una postergación deliberada de la complejidad de portabilidad hasta consolidar el gameplay base. Los controles táctiles requirieron una segunda iteración meses después, un patron de refinamiento sobre feedback real consistente con el ciclo ágil [5]. La persistencia se resolvió con un formato binario propio priorizando rendimiento y tamaño, y el editor de mapas, herramienta de creación de contenido no esencial para el MVP, fue lo último en incorporarse. La secuencia real (core, luego móvil, luego persistencia, luego editor) coincide con la priorización que Scrum habría prescrito. En este sprint se implementó la Entrega Continua y la actualización automática del sitio de GitHub Pages.

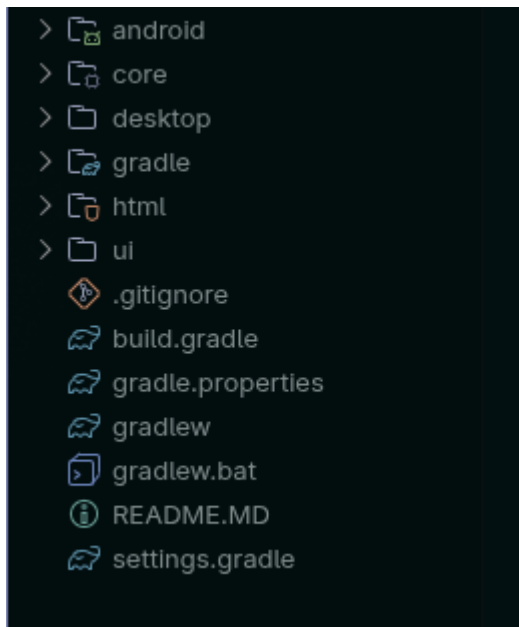


Fig. 6. Incorporación del módulo Android, sistema de persistencia.

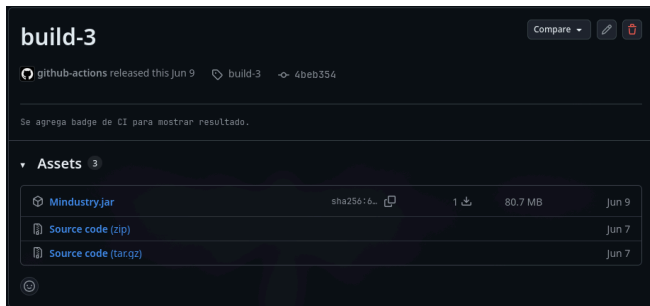


Fig 7. Generación automática del artefacto JAR.

C. Sprint 3: red, mods y ecosistema

La capa de red constituye el hallazgo más revelador. El sistema se construyó primero como una abstracción (interfaces ClientProvider y ServerProvider) antes de existir ninguna implementación concreta, un ejemplo directo de inversión de dependencias [18]. Esa abstracción permitió, dos años después, migrar de Kryonet a ArcNet sin reescribir la lógica de negocio dependiente de la red, y consolidar después el serializador como responsabilidad interna. La separación del módulo server como unidad de despliegue independiente (servidor headless) es un prerequisite para el CD. El sistema de mods evolucionó en capas (carga de JAR, contenido declarativo en JSON, scripting JavaScript y fork propio de Rhino), y la expansión a Steam, Discord e iOS se apoyó en una interfaz Platform que mantiene el núcleo agnóstico de cualquier plataforma externa. Estas particularidades son coherentes con lo reportado para el desarrollo de videojuegos [11], [12].

```
git log --all --graph --decorate --online --pretty=format:"%h - %ad : %s (%an)" -
--date=short -E --grep='kryonet|arcnet' -i
* 3e5ad07e8c - 2024-01-27 : Merge branch 'master' of https://github.com/Anuken/Mindustry
into async-ping (Anuken)
* 685e3cac83 - 2022-07-03 : Add support for SRV records (#6982) (Phinner)
* 8b9be6eafe - 2019-08-23 : Removed arcnet as dependency (Anuken)
* 01e1438382 - 2019-04-17 : Switched to ArcNet networking extension (Anuken)
* d562bfe52 - 2019-01-04 : Updated to next version of kryonet (Anuken)
* 9e25bacfdd - 2018-01-03 : Delete kryonet-release.jar (Anuken)
* 20ee3b385 - 2018-01-01 : Switched to different Kryonet fork; full Android support (Anu
ken)
* c7dbdd1773 - 2018-01-01 : Modularized Kryonet handler (Anuken)
* e24179cd4c - 2017-12-30 : Added full Kryonet server/client implementation (Anuken)
* f6e9710b33 - 2017-12-30 : Added basic Kryonet support (Anuken)
```

Fig. 8. Evolución del módulo de red y migración de Kryonet hacia ArcNet.

D. Sprint 4: calidad, release final y cierre

El sprint de cierre materializa el tercer nivel de madurez DevOps. El pipeline de CI se extiende con análisis de calidad y cobertura de pruebas mediante JaCoCo, con un umbral que interrumpe el flujo si no se alcanza; el umbral se fija tras analizar que la suite existente se concentra en lógica de contenido y parsing, evitando exigir cobertura sobre código de red o renderizado sin pruebas unitarias por su naturaleza. La Entrega Continua culmina en un release final con versionado semántico v1.0.0 que publica múltiples artefactos (JAR de escritorio, JAR de servidor y reporte de cobertura) y actualiza el sitio de documentación. Se aisló el workflow de release de producción en un archivo dedicado para evitar ejecuciones accidentales, una práctica DevOps recomendada [2], [6].

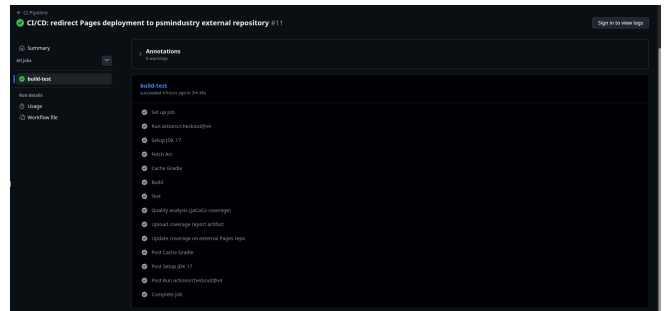


Fig. 9. Ejecución exitosa del pipeline CI #11 con compilación, pruebas y el paso de análisis JaCoCo completados.

E. Análisis comparativo del CI/CD upstream y académico

El análisis del repositorio original revela una automatización CI/CD funcional pero sin métricas formales de calidad: los workflows upstream compilan, ejecutan pruebas y automatizan el release, pero no incorporan análisis estático ni umbral de cobertura. Esta brecha es esperable en un proyecto de videojuego de código abierto, donde prima la velocidad de iteración [11], [12], y no constituye una crítica sino un punto de referencia real contra el cual contrastar las prácticas académicas. La introducción de cobertura formal sobre el fork es, en este sentido, un ejercicio pedagógico de lo que un proceso DevOps maduro añade sobre una base funcional preexistente [6].

VII. DISCUSIÓN Y LIMITACIONES

Los resultados sostienen la tesis del enfoque: la separación temprana del core, la inversión de dependencias en la red, la postergación de plataformas y herramientas no esenciales, y las refactorizaciones planificadas son decisiones que coinciden con una priorización implícitamente ágil. El factor determinante fue la abstracción temprana de interfaces, que actuó como habilitador de migraciones posteriores sin reescritura del núcleo, en coherencia con el principio de modularidad [18] y con la observación de que la refactorización arquitectónica reduce dependencias intermodulares [16]. Desde la perspectiva DevOps, el proyecto ilustra tres niveles de madurez (verificación, entrega y calidad) que un equipo real recorrerá de forma incremental.

El trabajo presenta varias limitaciones. Primero, se trata de una reconstrucción post hoc: el backlog no guió el desarrollo real, por lo que la coincidencia entre lo prescrito y lo observado debe interpretarse como evidencia de convergencia, no de causalidad. Segundo, es un estudio de caso único sobre un proyecto exitoso, lo que limita la generalización y podría introducir sesgo de supervivencia; las cautelas de la minería de repositorios sobre datos de plataformas de código abierto son aplicables [3]. Tercero, el umbral de cobertura se calibró para la naturaleza de la suite de pruebas existente y no representa una meta de calidad universal. Cuarto, la coexistencia entre el fork académico y el repositorio original generó colisiones de infraestructura en los workflows heredados, que debieron desactivarse para evitar errores de autenticación.

VIII. CONCLUSIÓN

Este trabajo mostró que la arqueología de repositorios, articulada como método del cangrejo, es un recurso válido para enseñar Scrum y DevOps sobre software real y maduro. La reconstrucción de un backlog de cuatro sprints y su contraste commit a commit con Mindustry evidencio que las decisiones técnicas reales convergen con las que la metodología ágil habría prescrito, y que la inversión temprana de dependencias fue la decisión que habilitó toda la evolución posterior sin reescritura del núcleo. En el plano DevOps, la infraestructura CI/CD construida sobre el fork alcanzó tres niveles de madurez y ofreció, mediante el análisis comparativo con el repositorio original, un caso de estudio concreto sobre cómo las prácticas académicas de calidad complementan una base funcional preexistente. Como trabajo futuro se plantea replicar el método sobre proyectos de distinto dominio para evaluar su generalidad y mitigar el sesgo de supervivencia.

AGRADECIMIENTOS

Los autores agradecen al Mg. Oscar Alberto Ramirez Valdez, docente del curso Ingeniería y Procesos de Software, por su orientación, y a la Escuela Profesional de Ingeniería de Sistemas de la Universidad Nacional de San Agustín de Arequipa por el apoyo brindado durante el desarrollo del proyecto.

REFERENCIAS

- [1] T. Dyba y T. Dingsoyr, "Empirical studies of agile software development: A systematic review," *Information and Software Technology*, vol. 50, n.º 9-10, pp. 833-859, 2008, doi: 10.1016/j.infsof.2008.01.006.
- [2] R. Jabbari, N. bin Ali, K. Petersen y B. Tanveer, "What is DevOps? A systematic mapping study on definitions and practices," en *Proc. Scientific Workshop Proc. XP2016*, Edinburgh, Reino Unido, 2016, art. 12, doi: 10.1145/2962695.2962707.
- [3] E. Kalliamvakou, G. Gousios, K. Blincoe, L. Singer, D. M. German y D. Damian, "An in-depth study of the promises and perils of mining GitHub," *Empirical Software Engineering*, vol. 21, n.º 5, pp. 2035-2071, 2016, doi: 10.1007/s10664-015-9393-5.
- [4] M. M. Lehman, "Programs, life cycles, and laws of software evolution," *Proceedings of the IEEE*, vol. 68, n.º 9, pp. 1060-1076, 1980, doi: 10.1109/PROC.1980.11805.
- [5] C. Verwijs y D. Russo, "A theory of Scrum team effectiveness," *ACM Transactions on Software Engineering and Methodology*, vol. 32, n.º 3, art. 74, 2023, doi: 10.1145/3571849.
- [6] A. Mishra y Z. Otaiwi, "DevOps and software quality: A systematic mapping," *Computer Science Review*, vol. 38, art. 100308, 2020, doi: 10.1016/j.cosrev.2020.100308.
- [7] M. Hilton, T. Tunnell, K. Huang, D. Marinov y D. Dig, "Usage, costs, and benefits of continuous integration in open-source projects," en *Proc. 31st IEEE/ACM Int. Conf. Automated Software Engineering (ASE)*, Singapur, 2016, pp. 426-437, doi: 10.1145/2970276.2970358.
- [8] B. Vasilescu, Y. Yu, H. Wang, P. Devanbu y V. Filkov, "Quality and productivity outcomes relating to continuous integration in GitHub," en *Proc. 10th Joint Meeting Foundations of Software Engineering (ESEC/FSE)*, Bergamo, Italia, 2015, pp. 805-816, doi: 10.1145/2786805.2786850.
- [9] H. Kagdi, M. L. Collard y J. I. Maletic, "A survey and taxonomy of approaches for mining software repositories in the context of software evolution," *Journal of Software Maintenance and Evolution: Research and Practice*, vol. 19, n.º 2, pp. 77-131, 2007, doi: 10.1002/smr.344.
- [10] I. Herraiz, D. Rodriguez, G. Robles y J. M. Gonzalez-Barahona, "The evolution of the laws of software evolution: A discussion based on a systematic literature review," *ACM Computing Surveys*, vol. 46, n.º 2, art. 28, 2013, doi: 10.1145/2543581.2543595.
- [11] E. Murphy-Hill, T. Zimmermann y N. Nagappan, "Cowboys, ankle sprains, and keepers of quality: How is video game development different from software development?," en *Proc. 36th Int. Conf. Software Engineering (ICSE)*, Hyderabad, India, 2014, pp. 1-11, doi: 10.1145/2568225.2568226.
- [12] L. Pascarella, F. Palomba, M. Di Penta y A. Bacchelli, "How is video game development different from software development in open source?," en *Proc. 15th Int. Conf. Mining Software Repositories (MSR)*, Gothenburg, Suecia, 2018, pp. 392-402, doi: 10.1145/3196398.3196418.
- [13] Z. Li, P. Avgeriou y P. Liang, "A systematic mapping study on technical debt and its management," *Journal of Systems and Software*, vol. 101, pp. 193-220, 2015, doi: 10.1016/j.jss.2014.12.027.
- [14] P. Kruchten, R. L. Nord y I. Ozkaya, "Technical debt: From metaphor to theory and practice," *IEEE Software*, vol. 29, n.º 6, pp. 18-21, 2012, doi: 10.1109/MS.2012.167.
- [15] W. N. Behutiye, P. Rodriguez, M. Oivo y A. Tosun, "Analyzing the concept of technical debt in the context of agile software development: A systematic literature review," *Information and Software Technology*, vol. 82, pp. 139-158, 2017, doi: 10.1016/j.infsof.2016.10.004.
- [16] M. Kim, T. Zimmermann y N. Nagappan, "An empirical study of refactoring challenges and benefits at Microsoft," *IEEE Transactions on Software Engineering*, vol. 40, n.º 7, pp. 633-649, 2014, doi: 10.1109/TSE.2014.2318734.
- [17] E. Murphy-Hill, C. Parnin y A. P. Black, "How we refactor, and how we know it," *IEEE Transactions on Software Engineering*, vol. 38, n.º 1, pp. 5-18, 2012, doi: 10.1109/TSE.2011.41.
- [18] D. L. Parnas, "On the criteria to be used in decomposing systems into modules," *Communications of the ACM*, vol. 15, n.º 12, pp. 1053-1058, 1972, doi: 10.1145/361598.361623.
- [19] K. Herzig, S. Just y A. Zeller, "The impact of tangled code changes on defect prediction models," *Empirical Software Engineering*, vol. 21, n.º 2, pp. 303-336, 2016, doi: 10.1007/s10664-015-9376-6.
- [20] G. Lacerda, F. Petrillo, M. Pimenta y Y. G. Gueheneuc, "Code smells and refactoring: A tertiary systematic review of challenges and observations," *Journal of Systems and Software*, vol. 167, art. 110610, 2020, doi: 10.1016/j.jss.2020.110610.
- [21] N. Rios, M. G. de Mendonca Neto y R. O. Spinola, "A tertiary study on technical debt: Types, management strategies, research trends, and base information for practitioners," *Information and Software Technology*, vol. 102, pp. 117-145, 2018, doi: 10.1016/j.infsof.2018.05.010.